

TP 10 — L'inventaire du magasin

Semaine 10 ■ Fichiers dans NSI/TP10/ ■ 2 h

Objectifs

- Construire une petite application autour d'un dictionnaire
- Écrire des fonctions qui modifient (ou non) leurs arguments — et le dire
- Observer alias et copies dans des cas concrets

Partie A — Expériences : alias et copies (console)

(20 min)

Prédire **par écrit** puis vérifier en console :

1. `a = [1, 2, 3]; b = a; a.append(4);` que vaut `b` ?
2. `c = list(a); a[0] = 99;` que vaut `c` ?
3. `a == c` ? `a is c` ? `a is b` ?
4. `grille = [[0] * 3] * 2; grille[0][0] = 1;` que vaut `grille` ? Surprise à noter — explication en semaine 12 !

Partie B — L'inventaire — `inventaire.py`

(60 min)

Le stock du magasin est un dictionnaire produit → quantité : `stock = {"cahier": 40, "stylo": 120, "gomme": 15}`.

Écrire, avec docstrings précisant si la fonction **modifie** le stock :

1. `vendre(stock, produit, n)` — retire `n` exemplaires (**modifie**) ; renvoie `True` si la vente a pu se faire (stock suffisant), `False` sinon (stock inchangé).
2. `recevoir(stock, produit, n)` — ajoute `n` exemplaires, en créant le produit s'il n'existe pas (**modifie**).
3. `en_rupture(stock)` — la **liste** des produits de quantité nulle (ne modifie pas ; compréhension bienvenue).
4. `total_articles(stock)` — le nombre total d'articles.
5. `fusion(s1, s2)` — renvoie un **nouveau** dictionnaire cumulant deux stocks, sans modifier ni `s1` ni `s2` (commencer par `resultat = dict(s1)` — la copie de dictionnaire).

Tester chaque fonction : au moins un cas normal et un cas limite (produit inconnu, quantité insuffisante).

Partie C — Le menu du magasin

(30 min)

Assembler une boucle de menu (motif de la semaine 6) :

```
(v) vendre    (r) recevoir    (l) lister    (q) quitter
```

L'option lister affiche chaque produit, sa quantité, et « RUPTURE » le cas échéant, avec des f-strings alignées : `f"{produit:<10} {quantite:>4}"`.

Partie D — Défi ★★

(10 min)

`inverse(d)` — renvoie le dictionnaire « retourné » : valeurs → listes de clés. Ex. `inverse({"a": 1, "b": 2, "c": 1})` → `{1: ["a", "c"], 2: ["b"]}` (réutiliser le motif de `occurrences`).